

Design Specification

1. Purpose

`cpp-nn-visualizer` is a C++17 desktop application for legal chess play and model introspection. The program has two goals:

1. Provide a complete human-vs-human chess board for Tutorial III and the final project demo.
2. Visualize how several chess evaluation families process the current position: NNUE, LC0 CNN, LC0 BT4, and a handcrafted Classical evaluator.

The program is intentionally split so chess rules and model inference can be tested without raylib, while the raylib layer only handles presentation and input.

2. Build Targets

The executable is assembled from two internal static libraries:

Target	Responsibility
<code>cnv_core</code>	Chess rules, game state, file I/O, tensor operations, NN loaders, NN forward passes, Classical evaluator, and MCTS.
<code>cnv_viz</code>	raylib UI, board rendering, controls, dialogs, activation views, editor, and tree visualization.
<code>cnv</code>	Thin executable entry point linking <code>cnv_viz</code> and <code>cnv_core</code> .
<code>cnv_tests</code>	Headless test binary run through CTest.

This split prevents raylib from leaking into the chess and NN modules.

3. Class Diagram

The high-level class diagram is generated from `docs/class-diagram.dot`.

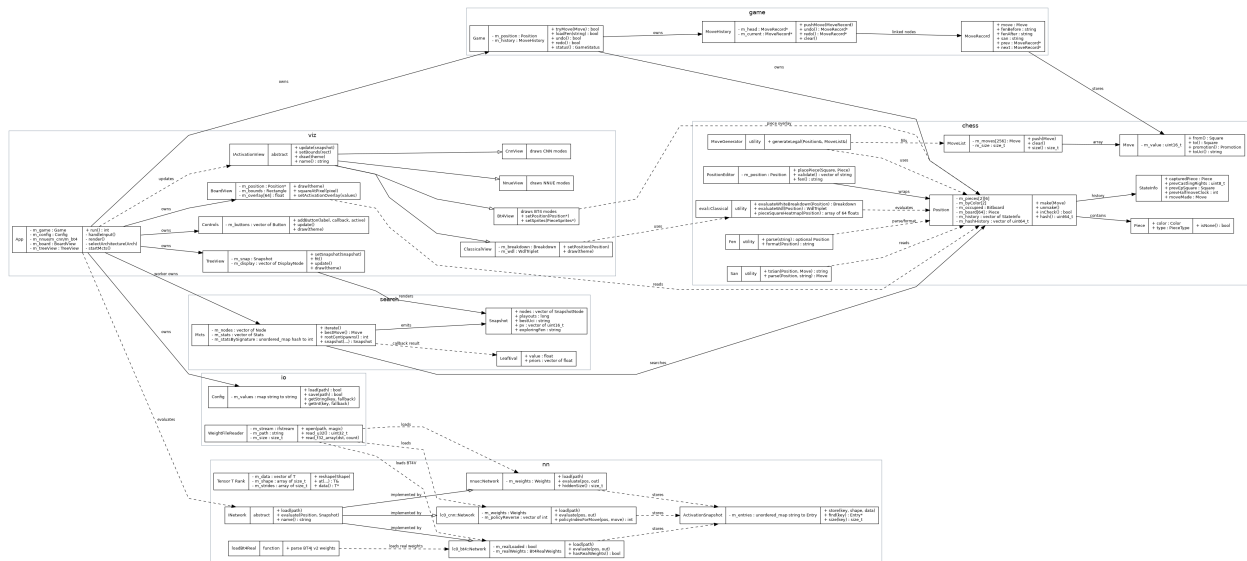


Figure 1: Core class diagram

4. Module Responsibilities

4.1 `src/chess`

The chess module owns all legal chess rules:

- **Position**: board state, side to move, castling, en-passant, clocks, make and unmake, check/draw helpers, and hash history.
- **Move**: packed UCI-style move representation.
- **MoveList**: fixed-capacity legal move container.
- **MoveGenerator**: legal move generation with check filtering.
- **Fen** and **San**: notation parsing/formatting.
- **PositionEditor**: mutable editor facade with validation.
- **Perft** and **Zobrist**: rule regression and hashing helpers.

The implementation uses bitboards for fast attacks and occupancy, plus a 64-square mailbox array for direct UI/parser lookup.

4.2 `src/chess/eval`

The Classical evaluator is a model-free C++ port of the chess-rtk handcrafted evaluator. It returns:

- an 11-term white-perspective centipawn breakdown,
- a side-to-move WDL triplet,
- per-piece piece-square tables,
- an occupied-square PST heatmap for the board overlay.

The visualizer uses this as its own activation choice and as a fallback evaluation path for search.

4.3 `src/game`

The game module wraps **Position** into a user-facing game:

- **Game**: legal move application, undo/redo, FEN loading, status, move list jumps, and game reset.
- **MoveHistory**: a manually implemented doubly linked list of move records.
- **Pgn**: PGN export helpers.

All normal board mutations go through **Game**, which keeps SAN, FEN snapshots, and undo/redo state consistent.

4.4 `src/io`

The I/O module contains:

- **Config**: flat key/value config loader and saver.
- **FenIo**: FEN file save/load helpers.
- **WeightFileReader**: little-endian binary reader for model blobs.

Runtime configuration is read once at startup from `config.ini`.

4.5 `src/nn`

The shared NN layer contains:

- **Tensor<T, Rank>**: rank-aware template tensor container.
- **ActivationSnapshot**: string-keyed activation tensor store.
- **INetwork**: polymorphic evaluator interface.
- **ops/**: CPU tensor operations such as convolution, matmul, attention, batch norm, layer norm, softmax, and activation functions.
- **CudaOps**: optional CUDA acceleration hooks; CPU remains the required path.

Each network writes named tensors into an `ActivationSnapshot`. The UI copies only the tensors it needs.

4.6 `src/nn/nnue`

The NNUE module implements:

- HalfKP feature encoding,
- accumulator refresh,
- compact NNUE binary loading,
- clipped activation and output contribution calculation,
- learned-weight atlas tensors for the NNUE panel.

The default file is `models/nnue-halfkp-demo.bin`.

4.7 `src/nn/lc0_cnn`

The CNN module implements:

- canonical LC0 112-plane input encoding,
- LC0J binary loading,
- residual trunk forward pass,
- policy head with compressed 4672-entry policy map,
- WDL/value head,
- final board heatmap and per-layer tensor snapshots.

The default file is `models/lc0-cnn-small-112p-4x32-policy4672-wd13.bin`. It has 4 residual blocks, 32 trunk channels, 73 policy planes, 4672 policy logits, and 3 WDL outputs.

4.8 `src/nn/lc0_bt4`

The BT4 module has two paths:

Path	Use
BT4J v2 real path	Loads compact CRTK BT4 weights and runs a native transformer forward pass.
BT4V synthetic fallback	Validates legacy metadata and generates deterministic transformer-like activations if no real BT4J model is loaded.

The default file is `models/lc0-bt4-tiny-96x4x4h.bin`. It is a compact real BT4J v2 model generated by `scripts/generate_small_bt4.py` with:

- 112 canonical input channels,
- 64 board tokens,
- 96-dimensional token embeddings,
- 4 encoder blocks,
- 4 attention heads,
- attention policy head to 1858 logits,
- WDL value head.

The loader and forward structs are written to support more BT4J components than the tiny default uses, including optional smolgen records and larger topologies. The official LC0 protobuf file format is not parsed at runtime.

4.9 src/search

Mcts is a single-threaded PUCT search engine used by the tree workbench. It is independent of raylib and accepts an evaluator callback:

```
using Evaluator =
    std::function<LeafEval(const Position&, const MoveList&)>;
```

The app supplies a callback that evaluates leaves through the selected architecture where possible. The search exposes bounded snapshots containing FEN, move, visits, prior, Q value, PV membership, and transposition signature for rendering.

4.10 src/viz

The visualization layer owns raylib resources and UI state:

- **App:** window lifetime, layout, input dispatch, model loading, async evaluation, search preview, MCTS worker, and rendering.
- **BoardView:** board, pieces, coordinates, selection, last move, legal targets, drag piece, and heatmap overlays.
- **Controls:** command button grid with active-state rendering.
- **NnueView, CnnView, Bt4View, ClassicalView:** architecture panels.
- **TreeView:** full-window MCTS graph with mini-board nodes.
- **FenInputDialog, PromotionPicker, EditorPalette, EditorPanel, MoveListView:** focused widgets for the chess workflow.

5. Major Data Members And Ownership

Class	Key data	Ownership / invariant
Position	bitboards, mailbox board, irreversible-state stack, hash history	Owns chess state; bitboards and mailbox are updated together.
Move	packed uint16_t value	Value type; castling/en-passant meaning is interpreted through Position.
MoveList	fixed array of up to 256 moves	Stack-friendly legal-move container with no heap allocation.
Game	Position, MoveHistory	Single mutation gateway for user-visible play.
MoveHistory	MoveRecord* head/current	Manually managed doubly linked list for undo/redo and move-list rendering.
ActivationSnapshot	map of string keys to tensor entries	Owns copied activation vectors produced by network evaluation.
Tensor<T, Rank>	std::vector<T>, shape, strides	Generic storage for tensor operations and tests.
lc0_cnn::Network	parsed Weights, policy reverse map	Owns model weights; reverse map is built lazily.
lc0_bt4::Network	Bt4RealWeights, real/synthetic flag	Uses real BT4J weights when loaded; otherwise synthetic fallback.
search::Mcts	node vector, stats vector, signature map	Owns tree state and emits render-friendly snapshots.
App	game, config, sprites, views, futures, threads, caches	Top-level owner of runtime state and raylib window lifetime.

Class	Key data	Ownership / invariant
<code>BoardView</code>	non-owning <code>Position*</code> , non-owning <code>PieceSprites*</code>	Renders the position owned by <code>Game</code> or an evaluation override.
<code>TreeView</code>	<code>search::Snapshot</code> , display nodes, parsed mini-boards	Owns display data derived from the latest MCTS snapshot.

6. Runtime Data Flow

Normal board mode:

mouse/keyboard input

```
-> App
-> Game / Position
-> async architecture evaluation
-> ActivationSnapshot
-> BoardView overlay + active activation view
-> raylib frame
```

Tree mode:

Start button

```
-> App::startMcts
-> search::Mcts worker thread
-> bounded search::Snapshot frames
-> TreeView layout
-> selected/followed FEN optional evaluation override
```

Search preview mode:

Search button / E

```
-> background alpha-beta style preview
-> selected architecture evaluator when available
-> current preview position
-> activation panel traces that preview position
```

7. Threading Model

The render loop stays on the main thread. Long-running work is moved off the frame path:

- network evaluation runs through a future and is guarded by a mutex when accessing network objects,
- live search preview runs in an async job with an atomic cancellation flag,
- MCTS tree growth runs in a worker thread and publishes snapshots under a mutex.

The UI copies completed results back into view-owned state before drawing. Search is stopped when the root position changes so stale search output is not applied to a different board.

8. File Formats

Format	Magic	Loader	Purpose
NNUE visualizer model	NNUE	<code>nnue::Loader</code>	Compact HalfKP feature, accumulator, and output weights.
LC0 CNN visualizer model	LC0J	<code>lc0_cnn::Loader</code>	Compact LC0-style CNN weights and policy map.

Format	Magic	Loader	Purpose
BT4 real visualizer model	BT4J	<code>lc0_bt4::loadBt4Real</code>	CRTK BT4J v2 transformer weights.
BT4 legacy visual metadata	BT4V	<code>lc0_bt4::Network::load</code>	Synthetic fallback dimension validation.
Config	text key/value	<code>io::Config</code>	Window, assets, models, startup architecture, MCTS, clock.
FEN	text	<code>Fen / FenIo</code>	Position import/export.

All binary model formats are little-endian.

9. UI Layout

The main window is split into:

1. a left board/status area,
2. a right command and activation panel area.

When an activation architecture is selected, the activation view receives most of the lower right panel and the move list becomes a compact footer. When Off is selected, the move list fills the available right panel space. Tree mode replaces the whole window with a toolbar and pan/zoom tree canvas.

10. Error Handling

- Missing NNUE weights fall back to a zero-ish visual path and show status.
- Missing CNN weights disable the real CNN path and use fallback search behavior.
- BT4 loads lazily. If the real BT4J file fails, the network remains usable via the synthetic fallback.
- Invalid FEN input remains in the dialog and displays the parse error.
- Invalid setup-editor positions cannot be applied.
- Search/evaluation exceptions are caught and fall back to Classical evaluation where possible.

11. Testing Strategy

The automated suite covers:

- bitboard helpers, piece/move encoding, legal move generation, perft, make/unmake, hashing, FEN, SAN, draw/status helpers,
- game loop, PGN export, move-history linked list, editor validation,
- config and FEN file I/O,
- tensor ops and activation snapshot storage,
- NNUE, CNN, BT4 loader/forward paths and runtime model smoke tests,
- Classical evaluator terms, WDL, and PST heatmaps,
- MCTS invariants and forced-mate behavior,
- top-level smoke test.

The current local run on 2026-06-08 passes 151 tests with 0 failures.

12. Intentional Limitations

- Human-vs-human play only; no engine opponent mode yet.
- FEN load/save is available; PGN import and file pickers are not implemented.
- Compact CNN and BT4 model files are untrained visualization models, so they demonstrate shape and data flow rather than playing strength.
- Official LC0 protobuf model import is outside the current runtime scope.

- UI tests are documented manually because the project uses an interactive desktop window rather than a headless browser UI.